14.4 多 Agent 路由

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~300,000 tokens，输出 ~24,000 tokens（本章合计）

单 Agent 模式下，所有消息都由同一个 Agent 处理。但实际使用中，用户可能希望不同的通道、账号甚至不同的聊天对象由不同的 Agent 来服务——例如一个"工作" Agent 处理 Slack 消息，一个"生活" Agent 处理 Telegram 消息，一个"编程" Agent 专门处理来自特定 Discord 频道的请求。OpenClaw 的 **多 Agent 路由**（Multi-Agent Routing）机制就是为此而设计的。

14.4.1 多 Agent 配置： `agents.list[]` 与绑定

多 Agent 的核心配置由两部分组成：**Agent 列表**（`agents.list`）和**绑定规则**（`bindings`）。

Agent 列表

Agent 列表定义了系统中存在哪些 Agent 以及各自的属性。每个 Agent 可以拥有独立的模型配置、技能过滤器、身份信息和沙箱策略：

```typescript
// src/config/types.agents.ts
export type AgentConfig = {
  id: string;            // Agent 唯一标识符
  default?: boolean;     // 是否为默认 Agent
  name?: string;         // 显示名称
  workspace?: string;    // 工作区目录
  agentDir?: string;     // Agent 状态目录
  model?: AgentModelConfig;  // 模型配置（主模型 + 回退列表）
  skills?: string[];     // 技能白名单（省略 = 全部技能）
  identity?: IdentityConfig; // 身份配置（名称/头像/主题色）
  groupChat?: GroupChatConfig; // 群聊配置
  subagents?: {          // 子 Agent 配置
    allowAgents?: string[];  // 允许派生的 Agent ID 列表
    model?: AgentModelConfig; // 子 Agent 默认模型
  };
  sandbox?: { ... };     // 沙箱隔离策略
  tools?: AgentToolsConfig;  // 工具权限配置
};

export type AgentsConfig = {
  defaults?: AgentDefaultsConfig;  // 全局默认配置
  list?: AgentConfig[];            // Agent 列表
};
```

一个典型的多 Agent 配置示例如下：

```yaml
# openclaw.yaml
agents:
  list:
    - id: home
      default: true
      name: "家庭助手"
      model: "anthropic/claude-sonnet-4-20250514"
      identity:
        name: "小克"
        emoji: "🏠"
    - id: work
      name: "工作助手"
      model:
        primary: "openai/gpt-4o"
        fallbacks: ["anthropic/claude-sonnet-4-20250514"]
      workspace: "~/work-projects"
      identity:
        name: "工作克"
        emoji: "💼"
    - id: code
      name: "编程助手"
      model: "anthropic/claude-sonnet-4-20250514"
      skills: ["code-review", "git", "terminal"]
      workspace: "~/dev"
```

默认 Agent 的解析

系统需要确定"默认 Agent"时， `resolveDefaultAgentId` 函数按以下优先级进行选择：

```ts
// src/agents/agent-scope.ts
export function resolveDefaultAgentId(cfg: OpenClawConfig): string {
  const agents = listAgents(cfg);
  if (agents.length === 0) {
    return DEFAULT_AGENT_ID;  // "main"
  }
  // 查找标记为 default=true 的 Agent
  const defaults = agents.filter((agent) => agent?.default);
  if (defaults.length > 1) {
    console.warn("Multiple agents marked default=true; using the first entry.");
  }
  const chosen = (defaults[0] ?? agents[0])?.id?.trim();
  return normalizeAgentId(chosen || DEFAULT_AGENT_ID);
}
```

解析逻辑如下：

1. 如果 `agents.list` 为空，返回常量 `DEFAULT_AGENT_ID` （值为 `"main"` ）
2. 如果有一个或多个 Agent 标记了 `default: true` ，取第一个
3. 如果没有 Agent 标记 `default` ，取列表中的第一个
4. 多个 `default: true` 会触发警告，但不会报错

> **衍生解释**： `normalizeAgentId` 函数执行字符串的 trim + lowercase 操作，确保 `"Home"` 、 `" home"` 和 `"HOME"` 都被视为同一个 Agent ID。这种标准化（Normalization）是分布式系统中避免标识符歧义的常见做法。

绑定类型定义

绑定（Binding）是将消息路由到特定 Agent 的规则：

```typescript
// src/config/types.agents.ts
export type AgentBinding = {
  agentId: string;      // 目标 Agent ID
  match: {
    channel: string;    // 通道标识（telegram/slack/discord 等）
    accountId?: string; // 账号 ID（同一通道可配置多个账号）
    peer?: {            // 对等方（聊天对象）
      kind: "dm" | "group" | "channel";
      id: string;
    };
    guildId?: string;   // Discord 服务器 ID
    teamId?: string;    // Slack 工作区 ID
  };
};
```

绑定配置示例：

```yaml
# openclaw.yaml
bindings:
  - agentId: work
    match:
      channel: slack
      teamId: "T01234567"
  - agentId: code
    match:
      channel: discord
      guildId: "123456789"
  - agentId: home
    match:
      channel: telegram
      accountId: personal
```

这三条绑定的含义是：来自 Slack 工作区 `T01234567` 的消息交给 `work` Agent 处理，来自 Discord 服务器 `123456789` 的消息交给 `code` Agent 处理，来自 Telegram `personal` 账号的消息交给 `home` Agent 处理。

14.4.2 基于通道/账号/对等方的路由规则

路由解析是多 Agent 系统中最关键的决策环节。 `resolveAgentRoute` 函数实现了一套**多级匹配**（Multi-Level Matching）策略，从最精确的匹配条件逐级回退到最宽泛的匹配。

路由输入与输出

路由解析的输入包含消息的完整来源信息：

```
// src/routing/resolve-route.ts
export type ResolveAgentRouteInput = {
  cfg: OpenClawConfig;
  channel: string;           // 通道标识
  accountId?: string | null; // 账号 ID
  peer?: RoutePeer | null;  // 聊天对等方
  parentPeer?: RoutePeer | null; // 父级对等方（用于线程继承）
  guildId?: string | null;  // Discord 服务器 ID
  teamId?: string | null;   // Slack 工作区 ID
};
```

路由输出包含解析后的 Agent ID 和会话键：

```
export type ResolvedAgentRoute = {
  agentId: string;           // 路由到的 Agent ID
  channel: string;           // 规范化后的通道
  accountId: string;         // 规范化后的账号 ID
  sessionKey: string;        // 会话键（用于持久化和并发控制）
  mainSessionKey: string;    // 主会话键（便捷别名）
  matchedBy:                 // 匹配方式（用于调试和日志）
    | "binding.peer"
    | "binding.peer.parent"
    | "binding.guild"
    | "binding.team"
    | "binding.account"
    | "binding.channel"
    | "default";
};
```

六级匹配优先级

`resolveAgentRoute` 的核心是一个优先级递减的匹配链：

```
// src/routing/resolve-route.ts（简化版）
export function resolveAgentRoute(input: ResolveAgentRouteInput): ResolvedAgentRoute {
  const channel = normalizeToken(input.channel);
  const accountId = normalizeAccountId(input.accountId);
  const peer = input.peer ? { kind: input.peer.kind, id: normalizeId(input.peer.id) } :

  // 第一步：预过滤——只保留 channel + accountId 匹配的绑定
  const bindings = listBindings(input.cfg).filter((binding) => {
    if (!matchesChannel(binding.match, channel)) return false;
    return matchesAccountId(binding.match?.accountId, accountId);
  });

  // 第二步：按优先级依次匹配
  // 优先级 1：精确对等方匹配
  if (peer) {
    const peerMatch = bindings.find((b) => matchesPeer(b.match, peer));
    if (peerMatch) return choose(peerMatch.agentId, "binding.peer");
  }

  // 优先级 2：父级对等方继承（线程场景）
  if (parentPeer && parentPeer.id) {
    const parentPeerMatch = bindings.find((b) => matchesPeer(b.match, parentPeer));
    if (parentPeerMatch) return choose(parentPeerMatch.agentId, "binding.peer.parent");
  }

  // 优先级 3：Discord 服务器（Guild）匹配
  if (guildId) {
    const guildMatch = bindings.find((b) => matchesGuild(b.match, guildId));
    if (guildMatch) return choose(guildMatch.agentId, "binding.guild");
  }

  // 优先级 4：Slack 工作区（Team）匹配
  if (teamId) {
    const teamMatch = bindings.find((b) => matchesTeam(b.match, teamId));
    if (teamMatch) return choose(teamMatch.agentId, "binding.team");
  }

  // 优先级 5：账号级别匹配（排除通配符、对等方、Guild、Team）
  const accountMatch = bindings.find((b) =>
    b.match?.accountId?.trim() !== "*" &&
    !b.match?.peer && !b.match?.guildId && !b.match?.teamId
  );
  if (accountMatch) return choose(accountMatch.agentId, "binding.account");
```

```
  // 优先级 6：通道级别匹配（accountId 为通配符 "*"）
  const anyAccountMatch = bindings.find((b) =>
    b.match?.accountId?.trim() === "*" &&
    !b.match?.peer && !b.match?.guildId && !b.match?.teamId
  );
  if (anyAccountMatch) return choose(anyAccountMatch.agentId, "binding.channel");

  // 兜底：使用默认 Agent
  return choose(resolveDefaultAgentId(input.cfg), "default");
}
```

下表总结了六级匹配优先级：

优先级	matchedBy	匹配条件	典型场景
1	`binding.peer`	channel + accountId + peer（kind + id）	特定用户/群组的专属 Agent
2	`binding.peer.parent`	channel + accountId + parentPeer	线程继承父消息的 Agent 绑定
3	`binding.guild`	channel + accountId + guildId	Discord 服务器级别绑定
4	`binding.team`	channel + accountId + teamId	Slack 工作区级别绑定
5	`binding.account`	channel + accountId（精确）	按通道账号绑定
6	`binding.channel`	channel + accountId= *	整个通道的默认 Agent
兜底	`default`	无匹配	使用系统默认 Agent

预过滤机制

匹配链的第一步不是直接遍历所有绑定，而是先进行**预过滤**：只保留通道和账号 ID 同时匹配的绑定。这意味着一条绑定只有在 `channel` 字段与消息来源通道一致时，才会进入后续的精细匹配。

账号 ID 的匹配有三种情况：

1. 绑定未指定 `accountId`（或为空）——只匹配默认账号（`DEFAULT_ACCOUNT_ID`）
2. 绑定指定 `accountId: "*"` ——匹配任意账号
3. 绑定指定具体 `accountId` ——精确匹配

线程继承

优先级 2 的"父级对等方继承"值得单独说明。在 Slack 和 Discord 中，用户可以在消息下方创建线程（Thread）。如果线程本身没有直接匹配到绑定规则，系统会检查线程所属的父消息（parentPeer）是否匹配某个绑定。这确保了"在某个频道下创建的线程"继承该频道的 Agent 分配。

Agent ID 验证

`choose` 辅助函数中调用了 `pickFirstExistingAgentId`，它会验证绑定中引用的 Agent ID 是否确实存在于 `agents.list` 中：

```
// src/routing/resolve-route.ts
function pickFirstExistingAgentId(cfg: OpenClawConfig, agentId: string): string {
  const normalized = normalizeAgentId(agentId);
  const agents = listAgents(cfg);
  if (agents.length === 0) return sanitizeAgentId(agentId);

  // 在 agents.list 中查找匹配的 Agent
  const match = agents.find((agent) => normalizeAgentId(agent.id) === normalized);
  if (match?.id?.trim()) return sanitizeAgentId(match.id.trim());

  // 未找到则回退到默认 Agent
  return sanitizeAgentId(resolveDefaultAgentId(cfg));
}
```

如果绑定引用了一个不存在的 Agent ID，系统不会报错，而是静默回退到默认 Agent。这是一种**防御性设计**——配置错误不应导致消息无法处理。

14.4.3 子 Agent 注册表（Subagent Registry）

除了静态的多 Agent 路由外，OpenClaw 还支持**动态子 Agent**（Subagent）机制：一个正在运行的 Agent 可以在运行时派生出子 Agent 来执行后台任务，任务完成后自动向父 Agent 汇报结果。

> **衍生解释**：子 Agent 模式类似于操作系统中的 `fork()` + `wait()` 模式——父进程（Parent Process）创建子进程执行特定任务，子进程完成后将结果返回给父进程。在 OpenClaw 中，父 Agent 通过 `sessions_send` 工具派生子 Agent，子 Agent 完成后通过"汇报流程"（Announce Flow）将结果注入父 Agent 的上下文。

SubagentRunRecord 数据模型

子 Agent 注册表是一个内存中的 `Map<string, SubagentRunRecord>` ，用于跟踪所有正在运行的子 Agent：

```
// src/agents/subagent-registry.ts
export type SubagentRunRecord = {
  runId: string;                // 运行唯一标识
  childSessionKey: string;      // 子 Agent 的会话键
  requesterSessionKey: string; // 父 Agent 的会话键
  requesterOrigin?: DeliveryContext;  // 父 Agent 的投递上下文
  requesterDisplayKey: string; // 父 Agent 显示名（日志用）
  task: string;                 // 任务描述
  cleanup: "delete" | "keep"; // 完成后是否删除子会话
  label?: string;               // 可选标签
  createdAt: number;            // 创建时间戳
  startedAt?: number;           // 开始执行时间戳
  endedAt?: number;             // 结束时间戳
  outcome?: SubagentRunOutcome; // 执行结果
  archiveAtMs?: number;         // 归档过期时间
  cleanupHandled?: boolean;     // 清理是否已处理
  cleanupCompletedAt?: number; // 清理完成时间
};
```

注册与生命周期

父 Agent 调用工具派生子 Agent 时， `registerSubagentRun` 函数被调用：

```typescript
// src/agents/subagent-registry.ts（简化版）
export function registerSubagentRun(params: {
  runId: string;
  childSessionKey: string;
  requesterSessionKey: string;
  requesterOrigin?: DeliveryContext;
  requesterDisplayKey: string;
  task: string;
  cleanup: "delete" | "keep";
  label?: string;
  runTimeoutSeconds?: number;
}) {
  const now = Date.now();
  const cfg = loadConfig();

  // 计算归档过期时间（默认 60 分钟后可清理）
  const archiveAfterMs = resolveArchiveAfterMs(cfg);
  const archiveAtMs = archiveAfterMs ? now + archiveAfterMs : undefined;

  // 计算等待超时
  const waitTimeoutMs = resolveSubagentWaitTimeoutMs(cfg, params.runTimeoutSeconds);

  // 注册到内存 Map
  subagentRuns.set(params.runId, {
    runId: params.runId,
    childSessionKey: params.childSessionKey,
    // ...其他字段
    createdAt: now,
    startedAt: now,
    archiveAtMs,
    cleanupHandled: false,
  });

  // 启动事件监听器
  ensureListener();
  // 持久化到磁盘
  persistSubagentRuns();
  // 启动归档清理定时器
  if (archiveAfterMs) startSweeper();

  // 通过 Gateway RPC 等待子 Agent 完成
```

```
  void waitForSubagentCompletion(params.runId, waitTimeoutMs);
}
```

这个函数做了四件关键的事：

1. **注册记录**——在内存 Map 中创建条目
2. **启动监听器**——订阅 Agent 生命周期事件，以便在子 Agent 结束时立即感知
3. **持久化**——将注册表写入磁盘，支持进程重启后恢复
4. **异步等待**——通过 Gateway 的 `agent.wait` RPC 方法等待子 Agent 执行完成

事件监听与完成检测

注册表通过两条路径检测子 Agent 的完成：

路径 1：进程内生命周期事件

```ts
// src/agents/subagent-registry.ts
function ensureListener() {
  if (listenerStarted) return;
  listenerStarted = true;
  listenerStop = onAgentEvent((evt) => {
    if (evt.stream !== "lifecycle") return;
    const entry = subagentRuns.get(evt.runId);
    if (!entry) return;

    if (evt.data?.phase === "start") {
      entry.startedAt = evt.data.startedAt;
      persistSubagentRuns();
      return;
    }
    if (evt.data?.phase === "end" || evt.data?.phase === "error") {
      entry.endedAt = evt.data?.endedAt ?? Date.now();
      entry.outcome = evt.data?.phase === "error"
        ? { status: "error", error: evt.data?.error }
        : { status: "ok" };
      persistSubagentRuns();

      // 触发汇报流程
      beginSubagentCleanup(evt.runId);
      void runSubagentAnnounceFlow({ /* ... */ });
    }
  });
}
```

路径 2：跨进程 RPC 等待

```ts
// src/agents/subagent-registry.ts（简化版）
async function waitForSubagentCompletion(runId: string, waitTimeoutMs: number) {
  const wait = await callGateway<{ status?: string; startedAt?: number; endedAt?: numbe
    method: "agent.wait",
    params: { runId, timeoutMs: waitTimeoutMs },
    timeoutMs: waitTimeoutMs + 10_000,
  });
  if (wait?.status === "ok" || wait?.status === "error") {
    // 更新记录并触发汇报
    const entry = subagentRuns.get(runId);
    if (entry) {
      entry.endedAt = wait.endedAt ?? Date.now();
      entry.outcome = wait.status === "error"
        ? { status: "error", error: wait.error }
        : { status: "ok" };
      persistSubagentRuns();
      void runSubagentAnnounceFlow({ /* ... */ });
    }
  }
}
```

双路径设计的原因是：子 Agent 可能在同一进程内以嵌入模式运行（路径 1 覆盖），也可能在独立进程中运行（路径 2 覆盖）。两条路径中只有一条会触发汇报——`beginSubagentCleanup` 函数通过 `cleanupHandled` 标志保证幂等性。

汇报流程（Announce Flow）

子 Agent 完成后，`runSubagentAnnounceFlow` 函数负责将结果注入父 Agent 的上下文：

```typescript
// src/agents/subagent-announce.ts（简化版）
export async function runSubagentAnnounceFlow(params: {
  childSessionKey: string;
  requesterSessionKey: string;
  requesterOrigin?: DeliveryContext;
  task: string;
  cleanup: "delete" | "keep";
  outcome?: SubagentRunOutcome;
  // ...其他字段
}): Promise<boolean> {
  // 1. 读取子 Agent 最后的回复内容
  const reply = await readLatestAssistantReply({
    sessionKey: params.childSessionKey,
  });

  // 2. 构建统计信息
  const statsLine = await buildSubagentStatsLine({
    sessionKey: params.childSessionKey,
    startedAt: params.startedAt,
    endedAt: params.endedAt,
  });

  // 3. 构建触发消息
  const triggerMessage = [
    `A background task "${taskLabel}" just ${statusLabel}.`,
    "", "Findings:", reply || "(no output)", "",
    statsLine, "",
    "Summarize this naturally for the user.",
  ].join("\n");

  // 4. 尝试队列化注入（如果父 Agent 正忙）
  const queued = await maybeQueueSubagentAnnounce({
    requesterSessionKey: params.requesterSessionKey,
    triggerMessage,
    requesterOrigin,
  });
  if (queued !== "none") return true;

  // 5. 直接发送给父 Agent
  await callGateway({
    method: "agent",
    params: {
      sessionKey: params.requesterSessionKey,
```

```
      message: triggerMessage,
      deliver: true,
      // ...通道信息
    },
  });

  // 6. 清理子会话（如果 cleanup="delete"）
  if (params.cleanup === "delete") {
    await callGateway({
      method: "sessions.delete",
      params: { key: params.childSessionKey, deleteTranscript: true },
    });
  }
}
```

汇报流程的关键步骤：

1. **读取子 Agent 回复**——从子会话的转录记录中读取最后一条 assistant 消息
2. **构建统计**——包括运行时长、Token 消耗、预估成本等
3. **构建触发消息**——将子 Agent 的输出和统计信息包装成一条发给父 Agent 的消息
4. **智能注入**——如果父 Agent 正在忙碌（有活跃的 Agent 循环），使用队列机制；否则直接发送
5. **清理**——根据 `cleanup` 策略，可选地删除子 Agent 的会话和转录

子 Agent 系统提示词

每个子 Agent 在创建时都会收到一个专用的系统提示词，明确告知它的角色和限制：

```ts
// src/agents/subagent-announce.ts
export function buildSubagentSystemPrompt(params) {
  return [
    "# Subagent Context",
    "",
    "You are a **subagent** spawned by the main agent for a specific task.",
    "",
    "## Your Role",
    `- You were created to handle: ${params.task}`,
    "- Complete this task. That's your entire purpose.",
    "- You are NOT the main agent. Don't try to be.",
    "",
    "## Rules",
    "1. **Stay focused** - Do your assigned task, nothing else",
    "2. **Complete the task** - Your final message will be automatically reported",
    "3. **Don't initiate** - No heartbeats, no proactive actions",
    "4. **Be ephemeral** - You may be terminated after task completion",
    "",
    "## What You DON'T Do",
    "- NO user conversations (that's main agent's job)",
    "- NO external messages unless explicitly tasked",
    "- NO cron jobs or persistent state",
    // ...
  ].join("\n");
}
```

这个系统提示词的设计体现了"最小权限原则"——子 Agent 只被授权完成指定任务，不能发起心跳、主动联系用户或创建持久化状态。

归档与清理

注册表内建了一个**清扫器**（Sweeper），定期检查已完成子 Agent 的过期状态：

```
// src/agents/subagent-registry.ts
async function sweepSubagentRuns() {
  const now = Date.now();
  for (const [runId, entry] of subagentRuns.entries()) {
    if (!entry.archiveAtMs || entry.archiveAtMs > now) continue;

    // 过期的记录：删除子会话并从注册表中移除
    subagentRuns.delete(runId);
    await callGateway({
      method: "sessions.delete",
      params: { key: entry.childSessionKey, deleteTranscript: true },
    });
  }
  if (subagentRuns.size === 0) stopSweeper();
}
```

清扫器每 60 秒运行一次。默认的归档过期时间为 60 分钟（可通过 `agents.defaults.subagents.archiveAfterMinutes` 配置）。当所有子 Agent 记录都已清理完毕，清扫器自动停止，避免不必要的定时器开销。

进程重启恢复

注册表支持持久化到磁盘和重启后恢复：

```
// src/agents/subagent-registry.ts
function restoreSubagentRunsOnce() {
  if (restoreAttempted) return;
  restoreAttempted = true;

  const restored = loadSubagentRegistryFromDisk();
  for (const [runId, entry] of restored.entries()) {
    if (!subagentRuns.has(runId)) {
      subagentRuns.set(runId, entry);
    }
  }
  // 恢复后，继续等待或汇报
  for (const runId of subagentRuns.keys()) {
    resumeSubagentRun(runId);
  }
}
```

`resumeSubagentRun` 会检查每条恢复的记录：如果子 Agent 已经结束但汇报尚未完成，立即触发汇报；如果子 Agent 仍在运行，重新启动 `agent.wait` 等待。

agents.list RPC 方法

Gateway 提供了 `agents.list` RPC 方法，供客户端查询当前配置的所有 Agent：

```
// src/gateway/server-methods/agents.ts
"agents.list": ({ params, respond }) => {
  const cfg = loadConfig();
  const result = listAgentsForGateway(cfg);
  respond(true, result, undefined);
};
```

`listAgentsForGateway` 函数收集 Agent 列表并附带身份信息（名称、头像 URL、主题色等），返回结构如下：

```
{
  defaultId: string;      // 默认 Agent ID
  mainKey: string;        // 主会话键
  scope: SessionScope;    // 会话作用域
  agents: Array<{
    id: string;
    name?: string;
    identity?: {
      name?: string;
      theme?: string;
      emoji?: string;
      avatar?: string;
      avatarUrl?: string;
    };
  }>;
}
```

客户端（如 Web 控制台和原生应用）通过这个 RPC 方法获取 Agent 列表，从而在 UI 中展示 Agent 选择器和对应的身份标识。

本节小结

1. **多 Agent 配置**由两部分组成： `agents.list` （Agent 定义）和 `bindings` （路由规则），每个 Agent 可以拥有独立的模型、技能、身份和沙箱配置。
2. **路由解析**采用六级优先级匹配——从精确的对等方匹配（peer）到通道级别匹配（channel），最后回退到默认 Agent，确保每条消息都能找到处理者。
3. **预过滤机制**先按 channel + accountId 筛选候选绑定，再进行精细匹配，避免无关绑定干扰路由结果。
4. **子 Agent 注册表**管理动态派生的子 Agent，通过双路径完成检测（进程内事件 + 跨进程 RPC）确保不遗漏任何完成通知。
5. **汇报流程**将子 Agent 结果自动注入父 Agent 上下文，支持队列化注入（父 Agent 忙碌时）和直接发送两种模式。
6. **持久化与恢复**机制保证进程重启不会丢失子 Agent 状态，归档清扫器自动回收过期记录。